

DiabEvidence: Project Report and Guide

Sep 28, 2026 · @Mano

I built DiabEvidence, a question-answering system that answers type 2 diabetes questions only from published research, and redesigned how it decides what to answer. After my changes it routes questions into scope correctly on 15 of 16 new questions, up from 7 in-scope misroutes on the original design, and it now tells users honestly when a valid question simply isn't covered by the literature.

What DiabEvidence is

DiabEvidence is a retrieval-augmented generation (RAG) system: it answers a question by first finding relevant passages in a fixed library of research, then writing an answer that cites only those passages. Diabetes is my test bed. The real contribution is the method for deciding *when* a system like this should answer and when it should refuse, which would carry over to any clinical field by swapping the library.

The library is 5,000 PubMed abstracts (reviews, meta-analyses and clinical trials from the last 10 years), split into 13,733 passages, plus 35 short patient-education passages from ADA and NIDDK for basic definitions. Each passage is tagged with its study population (type 1, type 2, gestational, prediabetes or mixed).

What happens to a question, in order:

1. **Scope check.** A small model reads only the question and decides whether it falls in DiabEvidence's four areas: diet and nutrition, glycemic control, body composition and weight, and diet-medication interactions. Clearly unrelated questions are refused here, before any search.
2. **Retrieval.** The question is turned into a vector with a local embedding model (all-MiniLM-L6-v2) and matched against the library in Pinecone. Definition questions search the patient-education tier; research questions search the study tier.
3. **Evidence audit.** A second check breaks the question into the specific claims it asks about and marks each retrieved passage as direct, partial or absent evidence for each claim.
4. **Routing.** Scope and evidence together decide whether to answer, answer with a caveat, or refuse, and which kind of refusal to give.

5. **Grounded answer.** If it answers, the main model writes one sentence at a time, each tied to the passages that support it, states which population the evidence comes from, and never gives personal dosing advice.

The backend is Python and FastAPI with direct Anthropic and Pinecone API calls (no LangChain, so every step is visible). The frontend is a single web page that streams the answer and shows the evidence and the judges' reasoning beside it.

How the project evolved

The project went through four big stages: a first prototype, a rebuild around trustworthiness, scaling the evidence base, and finally the redesign of how it decides to answer.

Date (2026)	Stage	What I did
Sep 28	Held-out evaluation	Ran 16 new questions through the final design; packaged all results
Sep 28	Revised scope rules	Rewrote the scope and evidence judge prompts around the question's outcome; re-ran the 43-question set
Sep 28	Two-stage judge	Split the answer/refuse decision into a scope check and an evidence audit; built the evaluation harness
Sep 28	Yes/no judge	Replaced the similarity cutoff with an LLM answerability judge
Sep 23–24	Question routing and UI	Split definition vs research questions across tiers; rebuilt the frontend and citation behaviour
Sep 13–17	Scale-up	Grew the library from 264 to 5,000 abstracts; froze a 43-question evaluation set with reference answers; tested a cross-encoder reranker
Sep 12–13	Trust features	Added abstention, contradiction surfacing, per-sentence citations and a patient-education tier
Sep 2–11	Rebuild	Dropped LangChain, moved to streaming answers and local embeddings to fit a free hosting tier

Date (2026)	Stage	What I did
May 21 – Jun 1	First prototype	A general health chatbot on Gemini and LangChain, deployed on Render

One lesson shaped everything after the scale-up: similarity scores measure how *related* a passage is, not whether it *answers* the question. A diabetes-adjacent question such as laser treatment for retinopathy scored as high as a genuinely in-scope one, so no cutoff could separate them. That finding is why the rest of this report is about judges rather than thresholds.

The problem I set out to fix

The yes/no answerability judge was better than a similarity cutoff, but my 43-question evaluation showed it made three kinds of mistakes.

- **It mixed up two different questions.** "Is this question in our subject area?" and "Did we find enough evidence to answer it?" were collapsed into one yes/no. So valid in-scope questions with thin search results were told they were *out of scope*, which is simply false.
- **It accepted passages that were only on the same topic.** Ids 1, 10 and 22 were judged answerable because the passages looked related, and the answers they produced were not supported by those passages. Of 13 answered in-scope questions, only 7 were fully supported.
- **It refused too much.** Ids 7 and 18 were refused outright when an answer with an honest caveat would have been more useful.

In a health setting both directions matter. Answering beyond the evidence is a safety problem. Refusing a reasonable question, or refusing it with the wrong explanation, makes the tool useless and misleading.

My redesign: a two-stage judge

I split the single yes/no gate into two independent checks, each answering one question only.

Stage A, the scope judge, sees the question and the scope charter, and nothing else. It never sees search results, so a good or bad search cannot bias whether a question counts as in scope. It returns in scope, adjacent (diabetes-related but outside the four areas) or

unrelated.

Stage B, the evidence judge, sees the question and the retrieved passages, without their similarity scores. It breaks the question into specific claims, marks each passage as direct, partial or absent evidence for each claim, checks whether the study populations match the question, and gives a verdict: sufficient, partial or insufficient. The key rule I wrote into it: *topical similarity is not evidence*. A passage about the same drug or nutrient doesn't count unless it actually states the claim.

The two verdicts together decide what the user sees:

Scope	Evidence	What the user gets
In scope	Sufficient	A normal answer
In scope	Partial	An answer limited to the directly supported passages, with a banner naming the gap
In scope	Insufficient	"This is a question DiabEvidence is meant to answer, but the literature in the index doesn't address it"
Adjacent	Sufficient	An answer with a note that it's outside the four core areas
Adjacent	Partial or insufficient	A refusal that lists the four areas covered
Unrelated	(not checked)	A refusal, with no search at all

The third row is the one I care most about. It separates "we don't cover this" from "we cover this, but the research we have doesn't answer it", which is the honest answer a clinician would give.

I also added safety behaviour around the answer itself: a population-mismatch banner when the evidence comes from a different group than the question asks about, a caveat when a comparison is stitched together from separate studies, and a personal-dosing flag. A question like "should I lower my insulin dose?" still gets the general evidence, but with a banner to confirm any change with a clinician, and the answer is forbidden from giving individual dosing advice.

How I revised in scope vs out of scope

The first two-stage run fixed the confusion between scope and evidence, but it exposed a new problem: the scope judge was reacting to *words* instead of *meaning*. It sent 7 of 24 in-scope questions to "adjacent" (ids 10, 12, 13, 16, 17, 20, 23) because they mentioned "exercise", "insulin" or "medication". For example, "Do GLP-1 drugs cause loss of lean body mass?" was filed under medication management, even though lean mass is body composition, one of the four core areas.

I went through the judge's reason for every refused question, then rewrote the scope rules and both judge prompts myself. The core change is one principle:

Scope is decided by the outcome the person wants to know about, not by the intervention the question names.

The revised scope judge works through fixed steps:

1. State the outcome in one short phrase, such as "post-meal blood glucose" or "lean muscle mass".
2. If the question is explicitly about type 1 or gestational diabetes, it is adjacent.
3. Map the outcome to one of the four areas. If it maps, the question is in scope, whether the intervention is a food, a drug, exercise or surgery.
4. Words like insulin, medication, exercise or surgery never make a question adjacent on their own.
5. Adjacent is only for diabetes questions whose outcome maps to none of the areas: eye, nerve, kidney or foot complications, mental health, devices, cost and access.
6. Set a separate safety flag if the question asks for a personal dose or medication change.
7. When unsure between in scope and adjacent, choose in scope, because the evidence check runs next and still blocks unsupported answers.

I also wrote explicit area definitions into the scope charter (`config/scope_charter.md`), for example that alcohol counts as diet and that insulin counts as a diabetes medication. The charter is the single editable source of truth: the judge reads it at startup, so changing scope never requires a code change.

For the evidence judge I made three changes: a passage counts as direct evidence in any population (a population mismatch is flagged separately rather than hiding the evidence), exactly one claim is marked as the core claim the question can't be answered without, and

a comparison built from two separate studies is allowed but always caveated.

Before running the revised design, I recorded my prediction in the repository: in-scope questions answered would rise from 10/24 to about 14/24.

How I evaluated it

I measured three things: whether each question is routed to the right scope, how often in-scope questions get an answer, and whether the answers given are actually supported by their cited passages.

Question sets.

- *Development set*: 43 frozen questions with reference answers: 24 in scope (6 per area), 17 diabetes-adjacent, 2 unrelated. The labels and questions were never edited during the work.
- *Held-out set*: 16 new questions (8 in scope, 5 adjacent, 3 unrelated), written after the design was frozen and checked so that none duplicates a development question.

Faithfulness grading. A separate grader model checks every answer sentence against the passages it cites and labels the answer supported, partially supported or unsupported. It uses a different model from the judges (Sonnet rather than Haiku) and shares no wording with the evidence judge's prompt, so the two don't make the same mistakes together. Every answered case is graded, not a sample, and each one is exported to a spreadsheet for human checking.

Statistics. Every rate comes with a Wilson 95% confidence interval, which behaves well at small sample sizes like mine.

Safeguards against fooling myself.

- Each evaluation runs once. I never adjusted a prompt after looking at its results on the same questions.
- Predictions were written down and committed before the runs.
- Automatic consistency checks (58 of them) confirm, for example, that every question has a valid result and that every answered case was graded.
- Every judge failure is recorded, not hidden. Three runs were interrupted (a parsing bug, low memory, a laptop restart) and one was cut short by an API usage limit. None had produced results before being restarted, and the fixes touched only plumbing, never the judge prompts.

- Before blaming the search step for two refusals, I searched the whole library directly. Ids 9 (dawn phenomenon) and 20 (alcohol with sulfonylureas) have no matching text anywhere, so those are true gaps in the literature, not search failures.

Results

The revised design routes almost every question to the right scope, answered 14 of 24 in-scope development questions (exactly my prediction), and held up on new questions for scope, though fewer of them could be answered from the library.

Scope routing

Design	Question set	Scope mistakes
Two-stage, original prompts	43 development	7 (in-scope questions sent to adjacent)
Two-stage, my revised prompts	43 development	1 (id 19)
Two-stage, my revised prompts	16 held-out	1 (id 107); all 5 adjacent and 3 unrelated correct

How often each design answers (95% Wilson intervals)

Design	In scope answered	Adjacent answered
Similarity cutoff (original)	79.2% (59.5–90.8)	47.1% (26.2–69.0)
Yes/no LLM judge	54.2% (35.1–72.1)	29.4% (13.3–53.1)
Two-stage, original prompts	41.7% (24.5–61.2)	0% (0–18.4)
Two-stage, revised prompts (development)	58.3% (14/24)	5.9% (1/17)
Two-stage, revised prompts (held-out)	37.5% (13.7–69.4)	0% (0–43.5)

A higher answer rate is not automatically better. The similarity cutoff answered the most, but it answered diabetes-adjacent questions nearly half the time and produced many unsupported answers. The goal is to answer what the evidence supports and refuse the rest for the right reason.

Faithfulness of the answers given

Design	Supported	Partially supported	Unsupported
Yes/no LLM judge (13 answered in scope)	7	6 (partial and unsupported not recorded separately)	
Two-stage, original prompts (10)	6	3	1
Two-stage, revised prompts, development (14 in scope)	6	5	3
Two-stage, revised prompts, held-out (3)	1	1	1

The revised design answers more questions, but the extra answers are mostly the "partial evidence" kind, and that is where the unsupported answers come from (ids 7, 10, 23 and held-out 108).

Answers vs refusals on the held-out set. Of the 8 in-scope questions, 3 were answered (105, 106, 108), 4 got the "valid question, no evidence in the library" response (sugary drinks, nuts, sleep, sulfonylureas vs DPP-4 inhibitors), and 1 was wrongly refused as out of scope (107, taking metformin with food). The personal-dosing flag fired correctly on 108.

Cost of the extra checks. On the original two-stage run, the scope check took about 1.1 s and the evidence audit about 3.6 s per question, but generation got about 10 s faster (26.0 s vs 36.6 s) because the model receives fewer, better-chosen passages. Average total time was 31.4 s.

What still doesn't work, and the limitations

The scope problem is essentially solved. The remaining weaknesses are in answer quality under partial evidence and in what the library contains.

Open problems

- **Partial-evidence answers overreach.** When the evidence only partly covers a question, the answer still sometimes claims more than the passages support. This is the most important thing to fix next, because it's the safety-relevant failure.
- **One remaining scope blind spot.** Food or nutrient questions about a medication where the outcome isn't blood sugar: "Does metformin affect vitamin B12?" (id 19) and "Should metformin be taken with food to reduce side effects?" (id 107). The judge

narrows "diet-medication interaction" to effects on glucose. A one-line clarification in the charter should fix it, but it needs new questions to test.

- **The evidence judge doesn't always apply the population rule.** For id 18 it still discounted prediabetes evidence that should count as direct evidence with a population banner.
- **Library gaps.** Most unanswered in-scope questions simply have no direct evidence in the 5,000 abstracts: the dawn phenomenon, grapefruit and diabetes drugs, nuts and fasting glucose. Only adding literature fixes these.

Limitations I state up front

- **Small samples.** 43 development and 16 held-out questions, with only 8, 5 and 3 per group in the held-out set. The intervals are wide, and most differences between designs are not statistically significant.
- **The development numbers are optimistic.** I revised the prompts after studying failures on those same 43 questions, so they measure whether the fixes worked on the cases that motivated them, not general performance.
- **The held-out set is only semi-independent.** The 16 questions were drafted with the AI assistant I used for implementation, at my request, after the design was frozen. Because the drafter had seen the failure cases, these results are not a clean held-out measurement. A set written independently by someone else is the obvious next step.
- **Faithfulness is judged by a model.** The grader is independent of the judges but is still an LLM, and it isn't fully consistent run to run. I've exported every graded answer for human grading, which hasn't been done yet.

Study guide: what I should be able to explain

If I can explain the ideas, files and questions below in my own words, I can defend the whole project.

Key ideas

Term	What it means in this project
RAG	Retrieve passages first, then generate an answer only from them
Embedding	A list of 384 numbers representing a text's meaning; similar texts get similar vectors

Term	What it means in this project
Cosine similarity	How close two embeddings are; measures relatedness, not whether a passage answers the question
Pinecone namespace	A separate partition of the vector index; <code>v2_5k</code> is the 5,000-abstract library
Study vs background tier	Research abstracts vs patient-education pages; background may only support definitions, never research claims
Scope charter	<code>config/scope_charter.md</code> : the editable definition of what's in scope, read by the scope judge
Stage A / Stage B	The scope judge (question only) and the evidence judge (question plus passages)
Direct / partial / absent	Whether a passage states a claim, touches the topic without stating it, or says nothing about it
Core proposition	The one claim a question can't be answered without; the evidence verdict hinges on it
Fail closed	On any error, refuse rather than answer; a broken judge can never let an unchecked answer through
Faithfulness	Whether every answer sentence is supported by the passages it cites
Wilson interval	A 95% confidence interval for a proportion that stays sensible at small sample sizes
Held-out set	Questions not used while designing, so they test generalisation rather than fit

Where things live

File	What it does
<code>src/ingest/run_ingest.py</code>	Fetches PubMed abstracts, chunks them, tags population, embeds and uploads to Pinecone

File	What it does
<code>src/rag/retriever.py</code>	Embeds a question and queries Pinecone
<code>src/rag/scope_judge.py</code>	Stage A: scope decision, reads the charter, caches results
<code>src/rag/evidence_judge.py</code>	Stage B: per-claim evidence audit
<code>src/rag/chain.py</code>	Runs the whole pipeline, applies the routing table, streams the answer
<code>src/api/main.py</code> , <code>src/api/index.html</code>	The web API and the single-page frontend
<code>config/scope_charter.md</code>	The scope rules I wrote
<code>eval/eval_set_v2.json</code> , <code>eval/heldout_v1.json</code>	The 43 development and 16 held-out questions
<code>scripts/run_eval_v2_twostage.py</code>	Runs an evaluation once, saving each result as it goes
<code>scripts/grade_faithfulness.py</code>	Grades every answered case and writes the hand-grading sheet
<code>eval/REPORT_twostage_judge.md</code>	The full technical report with every table
<code>eval/results/final_2026-09-28/</code>	The final result files and run logs

How to run it

1. Install: `python -m venv venv`, then `venv\Scripts\pip install -r requirements.txt`, and put the Anthropic and Pinecone keys in `.env`.
2. Start the app: `venv\Scripts\python -m uvicorn src.api.main:app`, then open the local address in a browser.
3. Run the tests: `venv\Scripts\python -m pytest tests/` (146 tests; one live test needs API credit).
4. Run an evaluation: `python -u -m scripts.run_eval_v2_twostage --namespace v2_5k --questions <file> --out <file>`, then the grader, then `scripts/report_revised_run.py`.

Questions I should be ready for

- *Why not just use a similarity threshold?* Because an off-topic diabetes question (retinopathy lasers) scored as high as an in-scope one. Similarity measures relatedness, not answerability.
- *Why does the scope judge never see the search results?* So that the quality of one search can't change whether a question counts as in scope. Scope is a property of the question.
- *Why is a lower answer rate not a failure?* Because the old designs answered things the evidence didn't support. I optimise for correct answers and honest refusals, not volume.
- *How do you know you didn't overfit the prompts?* I can't fully rule it out on the development set, which is why I report it as optimistic, recorded my prediction in advance, and ran a separate held-out set.
- *Why Wilson intervals?* With 8 or 17 questions per group, a normal approximation gives impossible ranges; Wilson intervals stay within 0–100% and are better calibrated.
- *What would you do next?* Tighten answers under partial evidence, clarify the diet-medication rule in the charter, get the answers human-graded, and build a larger independently written held-out set.

Tools and acknowledgements

I designed DiabEvidence and made its key decisions: the scope of the system, the two-stage judge design, the routing table, the outcome-based scope rules and the revised judge prompts, the evaluation protocol, and how the results are interpreted. I used Claude Code, an AI coding assistant, to implement much of the code from my specifications, run the evaluations and help draft this report.

- **Models:** Claude Haiku 4.5 for the scope and evidence judges; Claude Sonnet 5 for answers and faithfulness grading.
- **Retrieval:** Pinecone vector database; all-MiniLM-L6-v2 embeddings run locally with fastembed.
- **Evidence:** PubMed abstracts via NCBI E-utilities; patient-education pages from ADA and NIDDK.
- **Stack:** Python, FastAPI, vanilla HTML/CSS/JavaScript; hosted on Render.

DiabEvidence is a research prototype, not medical advice.